

数字电路设计

Digital Design and Computer Architecture: ARM[®]
Edition

Sarah L. Harris & David Money Harris

第 5 章详细学习笔记

Digital Building Blocks （数字构建模块）

涵盖第 5 章全部 138 页内容

日期：2026 年 6 月 15 日

考点总结 | 公式速查 | 概念解析 | 例题详解 | 备考指导

目录

1 章节概述 Chapter 5 Overview	3
2 算术电路 Arithmetic Circuits	3
2.1 1 位加法器 1-Bit Adders	3
2.1.1 半加器 Half Adder	3
2.1.2 全加器 Full Adder	4
2.2 多位加法器 Multibit Adders (CPAs)	5
2.2.1 行波进位加法器 Ripple-Carry Adder	5
2.2.2 先行进位加法器 Carry-Lookahead Adder (CLA)	5
2.2.3 前缀加法器 Prefix Adder	7
2.3 减法器 Subtractor	8
2.4 比较器 Comparator	8
2.4.1 相等比较器 Equality Comparator	8
2.4.2 小于比较器 Less Than Comparator	8
2.5 算术逻辑单元 ALU (Arithmetic Logic Unit)	8
2.5.1 ALU 基本功能	8
2.5.2 ALU 状态标志 ALU Status Flags	9
2.5.3 ALU 的 SystemVerilog 描述	10
2.6 移位器 Shifters	10
2.6.1 三种移位器类型	11
2.6.2 移位器的乘除应用	11
2.7 乘法器 Multipliers	11
2.7.1 乘法原理	12
2.8 除法器 Dividers	12
2.8.1 除法算法	12
3 数制系统 Number Systems	12
3.1 数的二进制表示概述	13
3.2 定点数 Fixed-Point Numbers	13
3.2.1 有符号定点数 Signed Fixed-Point	13
3.3 浮点数 Floating-Point Numbers	13
3.3.1 浮点数基本概念	14

3.3.2	IEEE 754 单精度浮点数 (32-bit)	14
3.3.3	浮点数转换示例	15
3.3.4	浮点数特殊值	15
3.3.5	浮点数精度	16
3.3.6	浮点数舍入 Rounding	16
3.3.7	浮点数加法	17
4	时序构建模块 Sequential Building Blocks	17
4.1	计数器 Counters	18
4.2	移位寄存器 Shift Registers	18
4.2.1	基本移位寄存器	18
4.2.2	带并行加载的移位寄存器	18
5	存储器阵列 Memory Arrays	19
5.1	存储器阵列概述	19
5.2	存储器位单元与字线	20
5.3	RAM 与 ROM 对比	20
5.4	DRAM 与 SRAM	20
5.4.1	DRAM (Dynamic RAM)	20
5.4.2	SRAM (Static RAM)	21
5.5	ROM 点表示法 Dot Notation	21
5.6	ROM 实现逻辑	22
5.7	查找表 LUT	22
5.8	多端口存储器 Multi-ported Memories	22
5.9	SystemVerilog 存储器阵列	23
6	逻辑阵列 Logic Arrays	23
6.1	PLA 与 FPGA 对比	23
6.2	PLA (可编程逻辑阵列)	24
6.3	FPGA (现场可编程门阵列)	24
6.3.1	FPGA 组成	24
6.3.2	逻辑单元 LE 组成	24
6.3.3	Altera Cyclone IV LE	25
6.3.4	LE 配置示例	25
6.3.5	FPGA 设计流程	25

7 考点总结与备考指南	26
7.1 核心公式速查	26
7.2 关键概念对比	26
7.3 高频考点分布	26
7.4 备考建议	27

章节概述 Chapter 5 Overview

本章是数字电路设计课程的核心章节，全面介绍数字系统中的各类 ** 构建模块（Building Blocks）**。这些模块体现了数字设计的三个基本原则：

- 层次化（Hierarchy）：复杂系统由更简单的组件层层构成
- 模块化（Modularity）：每个模块有明确定义的接口和功能
- 规则性（Regularity）：规则的结构便于扩展到不同规模

全书结构：第 5 章的构建模块将在第 7 章用于构建微处理器。 [p.3]

第 5 章六大主题（Topics）：

1. 算术电路（Arithmetic Circuits）

2. 数制系统（Number Systems）

3. 时序构建模块（Sequential Building Blocks）

4. 存储器阵列（Memory Arrays）

5. 逻辑阵列（Logic Arrays）

[p.2]

算术电路 Arithmetic Circuits

1 位加法器 1-Bit Adders

半加器 Half Adder

[p.4-6]

半加器接收两个 1 位输入 A 和 B ，产生和 S 与进位输出 C_{out} ：

- 和（Sum）： $S = A \oplus B$
- 进位输出（Carry out）： $C_{out} = AB$

表 1: 半加器真值表 (Half Adder Truth Table)

A	B	S	C_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

考点：半加器没有进位输入 (carry in)，只能处理两个 1 位数的加法，不能级联。

全加器 Full Adder

[p.4-7]

全加器接收三个 1 位输入 A 、 B 和进位输入 C_{in} ，产生和 S 与进位输出 C_{out} ：

- 和 (Sum): $S = A \oplus B \oplus C_{in}$
- 进位输出 (Carry out): $C_{out} = AB + AC_{in} + BC_{in}$

关键公式记忆：

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

表 2: 全加器真值表 (Full Adder Truth Table)

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

考点：全加器是最基本的加法单元，所有多位加法器都由它构成。 C_{out} 的产生条件是任意两个输入为 1 即可（三人表决逻辑）。

多位加法器 Multibit Adders (CPAs)

[p.8]

CPA（进位传播加法器，Carry Propagate Adder）的三种类型：

1. 行波进位加法器 Ripple-carry Adder（慢，硬件少）
2. 先行进位加法器 Carry-lookahead Adder（快，硬件多）
3. 前缀加法器 Prefix Adder（更快，硬件更多）

考点：三者的速度与硬件开销权衡（speed-area tradeoff）。

行波进位加法器 Ripple-Carry Adder

[p.9–12]

将 N 个 1 位全加器级联，进位从低位依次“波纹”传播到高位：

$$t_{ripple} = N \cdot t_{FA}$$

其中 t_{FA} 是单个全加器的延迟。

优点：硬件简单，面积小。

缺点：速度慢，延迟与位数 N 线性增长。

[p.10]

先行进位加法器 Carry-Lookahead Adder (CLA)

[p.13–26]

核心思想：同时计算所有列的进位，而不是等待进位逐位传播。

列信号 (Column Signals)

[p.14–18]

每列 i 定义两个信号：

- 产生 (Generate) G_i ：列 i 自己产生进位输出

$$G_i = A_i B_i$$

- 传播 (Propagate) P_i ：列 i 将进位输入传播到进位输出

$$P_i = A_i \oplus B_i$$

进位输出公式：

$$C_i = G_i + P_i \cdot C_{i-1}$$

块信号 (Block Signals)

[p.19–20]

对于 k -bit 块 (从位 j 到位 i), 定义块级产生和传播:

- 块传播 $P_{i:j} = P_i P_{i-1} P_{i-2} \cdots P_j$
- 块产生 $G_{i:j} = G_i + P_i(G_{i-1} + P_{i-1}(G_{i-2} + \cdots + P_j G_j))$
- 进位输出 $C_i = G_{i:j} + P_{i:j} \cdot C_{j-1}$

考点: 对于 4 位块 ($P_{3:0}$ 和 $G_{3:0}$):

$$P_{3:0} = P_3 P_2 P_1 P_0$$

$$G_{3:0} = G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0))$$

[p.20]

32 位 CLA with 4 位块

[p.21–24]

4 步加法过程:

1. 计算所有列的 G_i 和 P_i
2. 计算 k -bit 块的 $G_{i:j}$ 和 $P_{i:j}$
3. C_{in} 通过每块 propagate/generate 逻辑传播 (同时计算和)
4. 计算最高有效 k -bit 块的和

[p.23]

CLA 延迟

[p.26]

对于 N -bit CLA 使用 k -bit 块:

$$t_{CLA} = t_{pg} + t_{pg_block} + \left(\frac{N}{k} - 1\right) t_{AND_OR} + k \cdot t_{FA}$$

其中:

- t_{pg} : 产生所有 P_i 、 G_i 的延迟
- t_{pg_block} : 产生所有 $P_{i:j}$ 、 $G_{i:j}$ 的延迟
- t_{AND_OR} : k -bit CLA 块中最终 AND/OR 门从 C_{in} 到 C_{out} 的延迟

结论: 对于 $N > 16$, CLA 远快于行波进位加法器。

[p.26]

前缀加法器 Prefix Adder

[p.27–34]

核心思想：同时计算每一列的进位输入 C_{i-1} ，然后直接计算和：

$$S_i = (A_i \oplus B_i) \oplus C_{i-1}$$

计算 1 位、2 位、4 位、8 位等块级信号，直到所有 G_i （即 C_{i-1} ）已知。共有 $\log_2 N$ 个阶段。
[p.27]

前缀加法器公式

[p.28–29]

将列 -1 定义为 C_{in} ：

$$G_{-1} = C_{in}, \quad P_{-1} = 0$$

进位输入 $C_{i-1} = G_{i-1:-1}$ （跨越列 $i-1$ 到 -1 的产生信号）。

块合并公式：

$$G_{i:j} = G_{i:k} + P_{i:k} \cdot G_{k-1:j}$$

$$P_{i:j} = P_{i:k} \cdot P_{k-1:j}$$

含义：

- **产生：**块 $i:j$ 产生进位，如果上半部分 ($i:k$) 产生进位，或上半部分传播了下半部分 ($k-1:j$) 产生的进位
- **传播：**块 $i:j$ 传播进位，如果上半部分和下半部分都传播进位

[p.29]

前缀加法器延迟

[p.32]

$$t_{PA} = t_{pg} + \log_2 N \cdot t_{pg_prefix} + t_{XOR}$$

其中 t_{pg_prefix} 是黑色前缀单元（black prefix cell，AND-OR 门）的延迟。

三种加法器延迟对比（32 位）

[p.33–34]

假设：2 输入门延迟 = 100 ps，全加器延迟 = 300 ps，CLA 使用 4 位块：

表 3: 32 位加法器延迟对比

加法器类型	公式	延迟
Ripple-carry	$t = 32 \times 300 \text{ ps}$	9.6 ns
CLA (4-bit)	$t = 100 + 600 + 7 \times 200 + 4 \times 300 \text{ ps}$	3.3 ns
Prefix	$t = 100 + \log_2 32 \times 200 + 100 \text{ ps}$	1.2 ns

考点：前缀加法器最快（对数复杂度），CLA 次之，行波进位最慢（线性复杂度）。

减法器 Subtractor

[p.35–36]

实现方式：将减数取反后加 1（即取二进制补码），然后做加法：

$$Y = A - B = A + \overline{B} + 1$$

考点：减法器通过复用加法器实现，只需对 B 取反并将 C_{in} 设为 1。

比较器 Comparator

[p.37–39]

相等比较器 Equality Comparator

[p.37–38]

逐位比较 A 和 B 的对应位，使用 XNOR 门检查每一位是否相等，然后用 AND 门汇总。

考点： $Equal = \prod_{i=0}^{N-1} \overline{A_i \oplus B_i}$ （等价于 $\overline{A_i \oplus B_i}$ 的 AND）。

小于比较器 Less Than Comparator

[p.39]

计算 $A - B$ ，检查结果的最高位（符号位）：

$$A < B \quad \text{if} \quad (A - B)[N - 1] = 1$$

考点：利用减法器的符号位判断大小关系。

算术逻辑单元 ALU (Arithmetic Logic Unit)

[p.40–59]

ALU 基本功能

[p.40–47]

ALU 执行四种操作，由 2 位 ALUControl 信号选择：

表 4: ALU 功能表		
ALUControl[1:0]	功能	Function
00	加法	Add ($A + B$)
01	减法	Subtract ($A - B$)
10	按位与	AND ($A \& B$)
11	按位或	OR ($A B$)

考点：ALUControl[0] 同时控制：是否对 B 取反（决定加/减），以及 C_{in} 的值（加时 =0，减时 =1）。

ALU 状态标志 ALU Status Flags

[p.48-58]

表 5: ALU 状态标志		
标志	名称	含义
N	Negative（负）	结果为负（Result 最高位 =1）
Z	Zero（零）	结果为 0（所有位 =0）
C	Carry（进位）	加法器产生进位输出
V	oVerflow（溢出）	加法器溢出

N (Negative) : $N = Result[N - 1]$ （结果的最高有效位） [p.50]

Z (Zero) : $Z = 1$ 当结果的所有位为 0。使用缩减 NOR 运算。 [p.51]

C (Carry) : $C = 1$ 当加法器的 $C_{out} = 1$ 且 ALU 在执行加法或减法（ALUControl[1]=0）。 [p.52]

V (oVerflow) : 溢出检测条件： [p.53-57]

- 加法时：两个同号数相加产生异号结果
- 减法时：两个异号数相减（等价于同号相加），产生异号结果

溢出公式：

$$V = \overline{ALUControl_0} \cdot \overline{(A_{N-1} \oplus B_{N-1} \oplus ALUControl_0)} \cdot (A_{N-1} \oplus Sum_{N-1})$$

更简洁的形式：

$$V = (ALUControl_1 == 0) \&\& \sim(A[N-1] \text{ xor } B[N-1] \text{ xor } ALUControl_0) \&\& (A[N-1] \text{ xor } Sum[N-1])$$

考点：溢出的判断逻辑是 ALU 设计的高频考点。

ALU 的 SystemVerilog 描述

[p.59]

```
module alu #(parameter WIDTH = 3)(
    input  logic [WIDTH:0] a, b,
    input  logic [1:0]     ALUControl,
    output logic [WIDTH:0] Result,
    output logic [3:0]     ALUFlags);
    logic neg, zero, carry, overflow;
    logic [WIDTH:0]  condinvb;
    logic [WIDTH+1:0] sum;
    assign condinvb = ALUControl[0] ? ~b : b;
    assign sum = a + condinvb + ALUControl[0];
    always_comb
        casex (ALUControl[1:0])
            2'b0?: Result = sum;
            2'b10: Result = a & b;
            2'b11: Result = a | b;
        endcase
    assign neg      = Result[WIDTH];
    assign zero     = (Result == (WIDTH+1)'b0);
    assign carry    = (ALUControl[1] == 1'b0) & sum[WIDTH+1];
    assign overflow = (ALUControl[1] == 1'b0) &
        ~(a[WIDTH] ^ b[WIDTH] ^ ALUControl[0]) &
        (a[WIDTH] ^ sum[WIDTH]);
    assign ALUFlags = {neg, zero, carry, overflow};
endmodule
```

考点：理解 casex 的使用——ALUControl[0] 为 don't-care (x) 时进行加/减操作。

移位器 Shifters

[p.60–63]

三种移位器类型

[p.60–61]

1. 逻辑移位器 **Logical Shifter**: 移位时空位填 0

- 例: $11001 \gg 2 = 00110$ (右移填 0)
- 例: $11001 \ll 2 = 00100$ (左移填 0)

2. 算术移位器 **Arithmetic Shifter**: 右移时填旧 MSB (符号扩展), 左移同逻辑移位

- 例: $11001 \ggg 2 = 11110$ (右移填原符号位)
- 例: $11001 \lll 2 = 00100$ (左移同逻辑)

3. 循环移位器 **Rotator**: 移出的位从另一端移入

- 例: $11001 \text{ ROR } 2 = 01110$
- 例: $11001 \text{ ROL } 2 = 00111$

[p.60–61]

移位器的乘除应用

[p.63]

- 左移 = 乘法: $A \ll N = A \times 2^N$
 - 例: $00001 \ll 2 = 00100$ ($1 \times 4 = 4$)
 - 例: $11101 \ll 2 = 10100$ ($-3 \times 4 = -12$)
- 算术右移 = 除法: $A \ggg N = A \div 2^N$
 - 例: $01000 \ggg 2 = 00010$ ($8 \div 4 = 2$)
 - 例: $10000 \ggg 2 = 11100$ ($-16 \div 4 = -4$)

考点: 移位器实现乘除法 (2 的幂次倍), 是数字信号处理中的基本操作。

乘法器 **Multipliers**

[p.64–65]

乘法原理

- **部分积 (Partial Products):** 乘数的每一位与被乘数相乘
- 部分积移位后求和得到最终结果

十进制例: $230 \times 42 = 9660$ [p.64]

二进制例 (4×4 乘法器): [p.65]

- $A[3:0] \times B[3:0]$ 产生 8 位结果 $P[7:0]$
- $P = \sum_{i=0}^3 \sum_{j=0}^3 A_i B_j \cdot 2^{i+j}$

考点: 部分积的生成和累加是乘法器的核心。 $N \times N$ 乘法产生 $2N$ 位结果。

除法器 Dividers

[p.66–71]

除法算法

[p.70]

$A/B = Q + R/B$ (其中 Q 为商, R 为余数)

```
R' = 0
for i = N-1 to 0
    R = {R' << 1, A_i}    // 左移 R', 并附加被除数当前位
    D = R - B             // 试减
    if D < 0: Q_i = 0; R' = R    // 不够减, 商 0, 恢复余数
    else: Q_i = 1; R' = D      // 够减, 商 1
R = R'
```

考点: 二进制除法算法类似于手工长除法, 关键操作是”试减——判断——恢复”。

数制系统 Number Systems

[p.72–93]

数的二进制表示概述

[p.72]

- 正数：无符号二进制 (Unsigned binary)
- 负数：二进制补码 (Two's complement)、符号/数值 (Sign/Magnitude)
- 小数 (Fractions)：定点数 (Fixed-point) 和浮点数 (Floating-point)

定点数 Fixed-Point Numbers

[p.73-78]

概念：二进制小数点固定在某个位置 (隐含的)。

示例：6.75 使用 4 个整数位和 4 个小数位：

[p.74]

$$0110.1100 = 2^2 + 2^1 + 2^{-1} + 2^{-2} = 4 + 2 + 0.5 + 0.25 = 6.75$$

考点：整数位和小数位的位数必须事先约定一致。

有符号定点数 Signed Fixed-Point

[p.77-78]

以 -7.5_{10} 为例 (4 整数位 + 4 小数位)：

- 符号/数值 (Sign/Magnitude): 11111000
 - 最高位 = 1 表示负号，其余位为数值
- 二进制补码 (Two's Complement): 10001000

1. $+7.5 = 01111000$

2. 按位取反 = 10000111

3. 加 1 = 10001000

考点：定点补码的转换步骤与整数补码完全相同。

浮点数 Floating-Point Numbers

[p.79-93]

浮点数基本概念

[p.79]

二进制小数点”浮动”到最高有效 1 的右边，类似于科学记数法：

$$\pm M \times B^E$$

- M = 尾数 (Mantissa)
- B = 基数 (Base)
- E = 指数 (Exponent)

考点：浮点表示的三要素——尾数、基数、指数。

IEEE 754 单精度浮点数 (32-bit)

[p.80–83]



三个版本的演进：

[p.80–83]

1. 版本 1：直接存储：指数直接存储（无偏置），尾数全部存储（含前导 1）

- 例： $228_{10} = 11100100_2 = 1.11001 \times 2^7$
- 0 | 00000111 | 11100100000000000000000

[p.81]

2. 版本 2：隐含前导 1 (Implicit Leading 1)：23 位只存小数部分

- 因尾数第一位总是 1，无需存储（节省 1 位精度）
- 0 | 00000111 | 11001000000000000000000

[p.82]

3. 版本 3：偏置指数 (Biased Exponent)：bias = 127

- 偏置指数 = 127 + 实际指数
- $127 + 7 = 134 = 10000110_2$
- 0 | 10000110 | 11001000000000000000000

• = 0x43640000

[p.83]

IEEE 754 转换步骤（十进制 → 浮点）：

1. 十进制转二进制（不要颠倒步骤 1 和 2 的顺序！）

2. 写成二进制科学记数法： $1.xxx \times 2^E$

3. 符号位（正 =0, 负 =1）

4. 8 位偏置指数： $\text{bias}(127) + E$

5. 23 位小数（隐含前导 1，只存小数部分）

浮点数转换示例

[p.84-85]

例：将 -58.25_{10} 转为 IEEE 754 浮点数

[p.84-85]

1. 十进制 → 二进制： $58.25_{10} = 111010.01_2$
2. 科学记数法： 1.1101001×2^5
3. 符号位 = 1（负数）
4. 偏置指数 = $127 + 5 = 132 = 10000100_2$
5. 小数部分 = 110 1001 0000 0000 0000 0000

结果： 1 | 10000100 | 110100100000000000000000 = 0xC2690000

考点：这是典型的浮点数转换考题，必须熟练掌握。

浮点数特殊值

[p.86]

表 6: IEEE 754 特殊值			
数值	符号 Sign	指数 Exponent	小数 Fraction
0	X	00000000	全 0
∞ （无穷大）	0	11111111	全 0
$-\infty$ （负无穷大）	1	11111111	全 0
NaN（非数）	X	11111111	非零

考点： ∞ 和 NaN 的指数都是全 1 (11111111)，区别是 NaN 的小数非零。

浮点数精度

[p.87]

表 7: 单精度 vs 双精度

参数	单精度 Single	双精度 Double
总位数	32 bit	64 bit
符号位	1 bit	1 bit
指数位	8 bits	11 bits
小数位	23 bits	52 bits
偏置 bias	127	1023

考点：双精度的 bias = 1023，指数范围更大，精度更高。

浮点数舍入 Rounding

[p.88]

- 溢出 **Overflow**: 数值太大无法表示
- 下溢 **Underflow**: 数值太小无法表示
- 四种舍入模式:
 1. 向下舍入 (Down)
 2. 向上舍入 (Up)
 3. 向零舍入 (Toward zero)
 4. 向最近舍入 (To nearest) ——最常用

例：1.100101 (1.578125) 舍入到 3 位小数：

- Down: 1.100 (1.5)
- Up: 1.101 (1.625)
- Toward zero: 1.100 (1.5)
- To nearest: 1.101 (1.625 比 1.5 更接近 1.578125)

浮点数加法

[p.89–93]

8 步加法算法:

[p.89]

1. 提取指数和小数位 (Extract exponent and fraction bits)
2. 前置前导 1 形成尾数 (Prepend leading 1 to form mantissa)
3. 比较指数 (Compare exponents)
4. 必要时移位较小的尾数 (向大的对齐) (Shift smaller mantissa if necessary)
5. 尾数相加 (Add mantissas)
6. 规范化尾数并调整指数 (Normalize mantissa and adjust exponent)
7. 舍入结果 (Round result)
8. 将指数和小数重新组合为浮点格式 (Assemble back)

例: **0x3FC00000 + 0x40500000**

[p.90–93]

- $N1 = 0x3FC00000$: $S=0, E=127, F=.1 \Rightarrow$ 尾数 1.1
- $N2 = 0x40500000$: $S=0, E=128, F=.101 \Rightarrow$ 尾数 1.101
- 指数差: $127 - 128 = -1$, $N1$ 右移 1 位 $\Rightarrow 0.11 \times 2^1$
- 相加: $0.11 \times 2^1 + 1.101 \times 2^1 = 10.011 \times 2^1$
- 规范化: $10.011 \times 2^1 = 1.0011 \times 2^2$
- 结果: $S=0, E=2+127=129, F=.0011... \Rightarrow$ **0x40980000**

考点: 浮点加法中的对齐 (alignment) 是核心操作——向大指数对齐。

时序构建模块 Sequential Building Blocks

[p.94–96]

计数器 Counters

[p.94]

- 每个时钟边沿递增一次
- 循环计数: 000, 001, 010, 011, 100, 101, 110, 111, 000, 001...
- 应用:
 - 数字时钟显示 (Digital clock display)
 - 程序计数器 PC (Program Counter): 跟踪当前执行的指令

实现: N 位寄存器 + 加法器 (每次加 1)。

考点: 计数器的本质是寄存器 + 增量器。

移位寄存器 Shift Registers

[p.95–96]

基本移位寄存器

[p.95]

- 每个时钟边沿移入一个新位, 移出一个位
- 应用: 串行-并行转换器 (Serial-to-Parallel Converter)
 - 将串行输入 S_{in} 转为并行输出 $Q_{0:N-1}$

带并行加载的移位寄存器

[p.96]

- Load = 1: 作为普通 N 位寄存器 (并行加载)
- Load = 0: 作为移位寄存器
- 功能:
 - 串 $\rightarrow$ 并转换: $S_{in} \rightarrow Q_{0:N-1}$
 - 并 $\rightarrow$ 串转换: $D_{0:N-1} \rightarrow S_{out}$

考点: 移位寄存器的两种模式 (加载/移位) 及其在串并行转换中的应用。

存储器阵列 Memory Arrays

[p.97–126]

存储器阵列概述

[p.97–100]

功能：高效存储大量数据。

三种常见类型：

- DRAM（动态随机存取存储器，Dynamic RAM）
- SRAM（静态随机存取存储器，Static RAM）
- ROM（只读存储器，Read Only Memory）

基本结构：

[p.97–98]

- N 位地址， M 位数据
- 二维位单元（bit cell）阵列
- 2^N 行（深度 depth） $\times M$ 列（宽度 width）
- 阵列大小 = 深度 $\times$ 宽度 = $2^N \times M$

示例： $2^2 \times 3$ 位阵列

[p.99]

- 字数（Number of words）：4
- 字大小（Word size）：3 bits
- 地址 10 存储的是 100

名词对照：

- 深度（Depth）= 行数 = 字数 = 2^N
- 宽度（Width）= 列数 = 字大小 = M

存储器位单元与字线

[p.101–103]

- 字线 (Wordline): 类似使能信号，每次只有一条字线为 HIGH
- 位线 (Bitline): 数据通过位线读写
- 地址译码器: 将 N 位地址译码为 2^N 条字线之一

[p.103]

RAM 与 ROM 对比

[p.104–106]

表 8: RAM vs ROM		
特性	RAM	ROM
易失性	易失性 (Volatile): 断电丢失数据	非易失性 (Nonvolatile): 断电保留数据
读写速度	快速读写	快速读取, 写入不可能或很慢
典型应用	计算机主存 (DRAM)	相机、U 盘中的 Flash 存储器

[p.104–106]

考点: 易失性 vs 非易失性是基本概念区分。

DRAM 与 SRAM

[p.107–112]

DRAM (Dynamic RAM)

[p.107–110]

- 使用电容 (Capacitor) 存储数据位
- 动态 (Dynamic): 需要周期性刷新 (refresh) 和读后重写

- 电容漏电导致值衰减
- 读取操作会破坏存储值
- 发明者：Robert Dennard（1966 年，IBM） [p.108]
- 优势：结构简单，密度高，成本低

考点：DRAM 需要刷新是其称为”动态”的原因。

SRAM (Static RAM)

[p.107, 111–112]

- 使用交叉耦合反相器（Cross-coupled inverters）存储数据位
- 静态（Static）：只要供电就不需要刷新
- 速度更快，但面积更大，成本更高

对比考点：

表 9: DRAM vs SRAM 对比

特性	DRAM	SRAM
存储元件	电容	交叉耦合反相器
需要刷新	是	否
速度	较慢	较快
密度	高（1 个晶体管 + 电容）	低（6 个晶体管）
成本/位	低	高
应用	主存（Main memory）	高速缓存（Cache）

ROM 点表示法 Dot Notation

[p.113–115]

- 位线（bitline）和字线（wordline）交叉处有点 = 存储 1
- 无点 = 存储 0

考点：ROM 使用点表示法的硬件实现——交叉点有无晶体管/熔丝。

ROM 实现逻辑

[p.116–121]

ROM 可以用作查找表 (Lookup Table, LUT):

原理: 将输入 (地址) 映射到输出 (数据), 每个地址存储对应输入组合的输出值。 [p.116–119]

例: 用 $2^2 \times 3$ ROM 实现: [p.117–118]

- $X = AB$
- $Y = A + B$
- $Z = \overline{A \oplus B}$ (即 $A \odot B$, XNOR)

查找表 LUT

[p.122–123]

概念: 查找表 (Lookup Table) 根据输入组合 (地址) 查找对应的输出。

示例: 4 word $\times$ 1 bit 阵列实现 $Y = AB$: [p.122]

表 10: LUT 实现 $Y = AB$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

考点: LUT 是 FPGA 的基本逻辑单元, k 输入 LUT 可以实现任意 k 变量逻辑函数。

多端口存储器 Multi-ported Memories

[p.124]

- **端口 (Port):** 地址/数据对
- **3 端口存储器:**
 - 2 个读端口 ($A1/RD1$, $A2/RD2$)
 - 1 个写端口 ($A3/WD3$, $WE3$ 使能写)

- 寄存器文件 (Register File): 小型多端口存储器，用于 CPU 中

考点：寄存器文件是微处理器数据通路的核心组件（第 7 章将用到）。

SystemVerilog 存储器阵列

[p.125–126]

```
// 256 x 3 memory module with one read/write port
module dmem(
    input  logic      clk, we,
    input  logic [7:0] a,
    input  logic [2:0] wd,
    output logic [2:0] rd);
    logic [2:0] RAM[255:0];
    assign rd = RAM[a];           // 组合逻辑读
    always @(posedge clk)
        if (we)
            RAM[a] <= wd;        // 时序逻辑写
endmodule
```

考点：组合读 + 时序写是典型的同步存储器模式。

逻辑阵列 Logic Arrays

[p.127–138]

PLA 与 FPGA 对比

[p.127]

表 11: PLA vs FPGA		
特性	PLA	FPGA
结构	AND 阵列 + OR 阵列	逻辑单元 (LE) 阵列
逻辑类型	仅组合逻辑	组合逻辑 + 时序逻辑
内部连线	固定	可编程

PLA（可编程逻辑阵列）

[p.128–129]

结构：AND 阵列 $\rightarrow$ OR 阵列，两层可编程连接。

示例：

[p.128]

- $X = \overline{A}BC + A\overline{B}C$
- $Y = AB$

点表示法：AND 阵列和 OR 阵列中，交叉点有点 = 连接。

考点：PLA 通过编程 AND 阵列（产生乘积项 implicants）和 OR 阵列（求和）实现任意与或式逻辑。

FPGA（现场可编程门阵列）

[p.130–138]

FPGA 组成

[p.130–131]

- LE (Logic Element): 执行逻辑功能
- IOE (Input/Output Element): 与外部世界接口
- 可编程互连 (Programmable Interconnection): 连接 LEs 和 IOEs
- 部分 FPGA 还包含乘法器和 RAM 等专用模块

逻辑单元 LE 组成

[p.132]

每个 LE 包含：

- LUT（查找表）：执行组合逻辑
- Flip-flop（触发器）：执行时序逻辑
- Multiplexer（多路选择器）：连接 LUT 和触发器

Altera Cyclone IV LE

[p.133–134]

- 1 个四输入 LUT
- 1 个寄存器输出 (registered output)
- 1 个组合逻辑输出 (combinational output)

LE 配置示例

[p.136–137]

用 Cyclone IV LE 实现：

- $X = \overline{A}BC + A\overline{B}C$
- $Y = AB$

使用两个 LE：

- **LE 1:** LUT 配置为实现 X (3 输入函数，真值表映射)
- **LE 2:** LUT 配置为实现 Y (仅用 2 输入，其余输入为 don't-care)

考点：LUT 通过真值表编程实现任意组合逻辑函数。don't-care 条件的利用可简化配置。

FPGA 设计流程

[p.138]

使用 EDA 工具 (如 Xilinx Vivado)：

1. **设计输入 (Design Entry):** 使用原理图或 HDL
2. **仿真 (Simulate):** 验证设计
3. **综合与映射 (Synthesize & Map):** 综合设计并映射到 FPGA
4. **下载配置 (Download):** 将配置下载到 FPGA
5. **测试 (Test):** 验证硬件功能

考点：FPGA 设计流程中综合 (synthesis) 和映射 (mapping) 是核心步骤。

考点总结与备考指南

核心公式速查

表 12: 第 5 章核心公式速查表	
内容	公式
半加器	$S = A \oplus B, C_{out} = AB$
全加器	$S = A \oplus B \oplus C_{in}, C_{out} = AB + AC_{in} + BC_{in}$
列产生/传播	$G_i = A_i B_i, P_i = A_i \oplus B_i$
进位输出	$C_i = G_i + P_i \cdot C_{i-1}$
块产生	$G_{i:j} = G_i + P_i(G_{i-1} + P_{i-1}(G_{i-2} + \cdots + P_j G_j))$
块传播	$P_{i:j} = P_i P_{i-1} P_{i-2} \cdots P_j$
前缀合并	$G_{i:j} = G_{i:k} + P_{i:k} \cdot G_{k-1:j}, P_{i:j} = P_{i:k} \cdot P_{k-1:j}$
行波进位延迟	$t_{ripple} = N \cdot t_{FA}$
前缀延迟	$t_{PA} = t_{pg} + \log_2 N \cdot t_{pg_prefix} + t_{XOR}$
减法器	$Y = A - B = A + \overline{B} + 1$
左移 = 乘	$A \ll N = A \times 2^N$
右移 = 除	$A \gg N = A \div 2^N$
IEEE 754 偏置	bias = 127 (单精度), bias = 1023 (双精度)

关键概念对比

表 13: 关键概念对比表			
对比维度	选项 A	选项 B	选项 C
加法器速度	Ripple-carry (线性)	CLA (介于中间)	Prefix (对数)
RAM 类型	DRAM (电容,需刷新)	SRAM (反相器, 无需刷新)	—
存储器类型	RAM (易失)	ROM (非易失)	—
逻辑阵列	PLA (仅组合)	FPGA (组合 + 时序)	—
数制	定点 (位置固定)	浮点 (位置浮动)	—
移位器	逻辑 (填 0)	算术 (填符号位)	循环 (轮转)

高频考点分布

1. 全加器设计 (p.4–7): 真值表、逻辑表达式、级联方式

2. 先行进位加法器 (p.13–26): 产生/传播信号、块信号、延迟计算
3. 前缀加法器 (p.27–34): 对数级联结构、前缀合并公式、延迟对比
4. ALU 设计 (p.40–59): 功能表、状态标志 (N/Z/C/V)、溢出判断逻辑
5. 移位器 (p.60–63): 三种移位器区别、乘除法应用
6. IEEE 754 浮点数 (p.79–93): 格式转换、特殊值、加法算法
7. 存储器阵列 (p.97–126): DRAM vs SRAM、ROM 实现逻辑、LUT 概念
8. FPGA/PLA (p.127–138): 结构组成、LE 配置、设计流程

备考建议

- 熟练掌握全加器到多位加法器的演进逻辑 (Ripple $\rightarrow$ CLA $\rightarrow$ Prefix)
- 理解”产生/传播”范式——这是从 CLA 到 Prefix 的统一框架
- IEEE 754 浮点数转换步骤必须烂熟于心 (十进制 $\rightarrow$ 二进制 $\rightarrow$ 规范化 $\rightarrow$ 偏置指数)
- 浮点加法 8 步算法——对齐 (alignment) 是核心
- ALU 溢出判断是高频难点: 区分加法溢出 (同号相加得异号) 和减法溢出 (异号相减)
- 存储器的层次化理解: bit cell $\rightarrow$ array $\rightarrow$ LUT $\rightarrow$ FPGA LE
- PLA 和 FPGA 的结构对比: AND/OR 阵列 vs LUT+FF+ 互连